

Data Storage (continued)

Lecture 3

Data Storage (continued)

- Hexidecimal and Octal Systems
- Storing Text, Image and Sound
- Binary System
- Storing Integers
- Storing Fractions
- Data Compression
- Error Detection and Correction

Hexadecimal Notation

- **Hexadecimal notation** is Radix-16 numerical system.
- A shorthand notation for long bit patterns
 - Divides a pattern into groups of four bits each
 - Represents each group by a single symbol
- Example: 10100011 becomes A3

The hexadecimal coding system

Bit pattern	Hexadecimal representation
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Other Radix-N systems

Binary	Octal	Hexadecimal	Decimal
0000	0	0	0
0001	1	1	1
0010	2	2	2
0011	3	3	3
0100	4	4	4
0101	5	5	5
0110	6	6	6
0111	7	7	7
1000	10	8	8
1001	11	9	9
1010	12	A	10
1011	13	B	11
1100	14	C	12
1101	15	D	13
1110	16	E	14
1111	17	F	15

Representing Text

- Each character (letter, punctuation, etc.) is assigned a unique bit pattern.
 - **ASCII**: Uses patterns of 7-bits to represent most symbols used in written English text

01001000	01100101	01101100	01101100	01101111	00101110
H	e	l	l	o	.

- ISO developed a number of 8 bit extensions to ASCII, each designed to accommodate a major language group
- **Unicode**: Uses patterns up to 21-bits to represent the symbols used in languages world wide, 16-bits for world's commonly used languages
- **UTF-8 (UTF-16, UTF-32)**: Uses variable number of bytes per character. Few of the character bits used to determine the length of the character in bytes.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Unicode 16-bit Encoding

C0 Controls and Basic Latin																
U+	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
000x	[NUL]	[SOH]	[STX]	[ETX]	[EOT]	[ENQ]	[ACK]	[BEL]	[BS]	[HT]	[LF]	[VT]	[FF]	[CR]	[SO]	[SI]
001x	[DLE]	[DC1]	[DC2]	[DC3]	[DC4]	[NAK]	[SYN]	[ETB]	[CAN]	[EM]	[SUB]	[ESC]	[FS]	[GS]	[RS]	[US]
002x	[SP]	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
003x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
004x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
005x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
006x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
007x	p	q	r	s	t	u	v	w	x	y	z	{		}	~	[DEL]

[More Here...](#)

UTF-8 Encoding (variable-width)

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0xxxxxxx			
U+0080	U+07FF	110xxxxx	10xxxxxx		
U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx	
U+10000	^[nb 2] U+10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

1,112,064
characters
can be
encoded

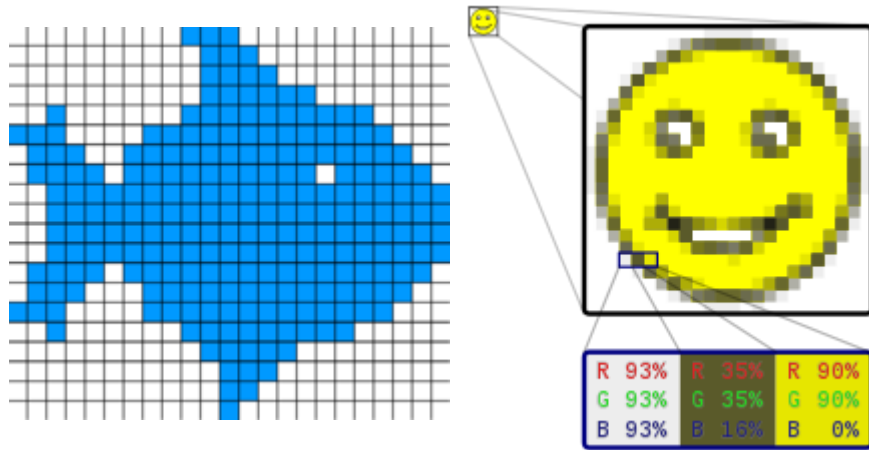
Examples of UTF-8 encoding

Character	Binary code point	Binary UTF-8	Hex UTF-8
\$ U+0024	010 0100	00100100	24
£ U+00A3	000 1010 0011	11000010 10100011	C2 A3
₹ U+0939	0000 1001 0011 1001	11100000 10100100 10111001	E0 A4 B9
€ U+20AC	0010 0000 1010 1100	11100010 10000010 10101100	E2 82 AC
한 U+D55C	1101 0101 0101 1100	11101101 10010101 10011100	ED 95 9C
⊙ U+10348	0 0001 0000 0011 0100 1000	11110000 10010000 10001101 10001000	F0 90 8D 88

Representing Images

- Bit map techniques
 - Pixel: short for “picture element”
 - smallest dot in picture representing a color
 - black-white pixel can be represented by single bit
 - RGB pixel can be represented by 3 bytes.
 - Luminance(white brightness)+chrominance (difference between white and (red, blue) color).
 - Gray image can be reproduced with only luminance.
 - Disadvantages: zooming worsens quality, can grow big in size
- Vector techniques
 - Different shapes encoded using analytic geometry
 - TrueType – geometrically describes text characters developed by Microsoft and Apple

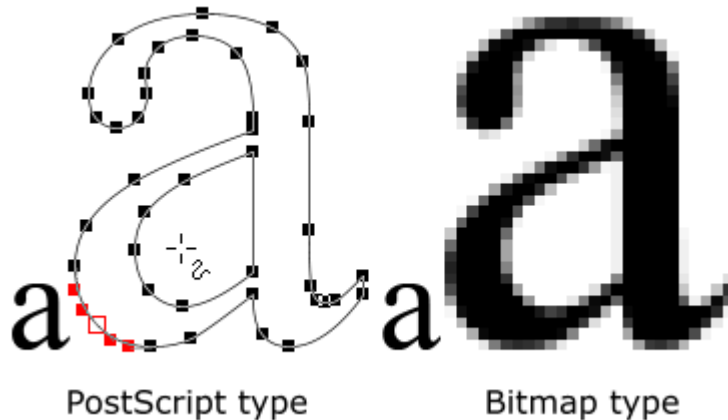
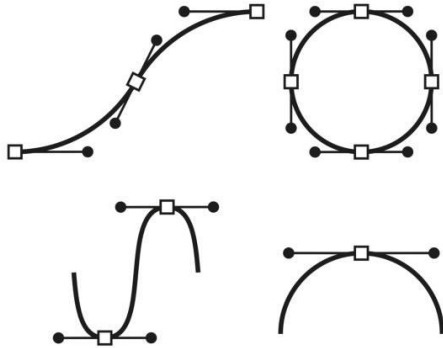
Raster Graphics



When viewed in full size, this image contains about 16 million pixels, each corresponding to a different color in the full set of RGB colors.

- Represents a 2D picture as a rectangular matrix of pixels.
- Pixel is the smallest element of raster image represented by range of colours it can represent
- More bits are allocated per pixel, more colours it can represent
 - 16-bit pixel can represent 65,536 colours (HighColor)
 - 24-bit pixel can represent around 16,777,216 colours (TrueColor)
- Human eye can distinguish from 1 million to 10 million different colours
- Resolution is the physical size of the pixel
 - dots-per-inch (dpi) is used to indicate printer's resolution
 - pixels-per-inch (ppi) is used to indicate screen's resolution
- Commonly used in *digital photography* and *remote sensing*
- File formats: *JPEG, GIF, PNG, TIFF, BMP*

Vector Graphics

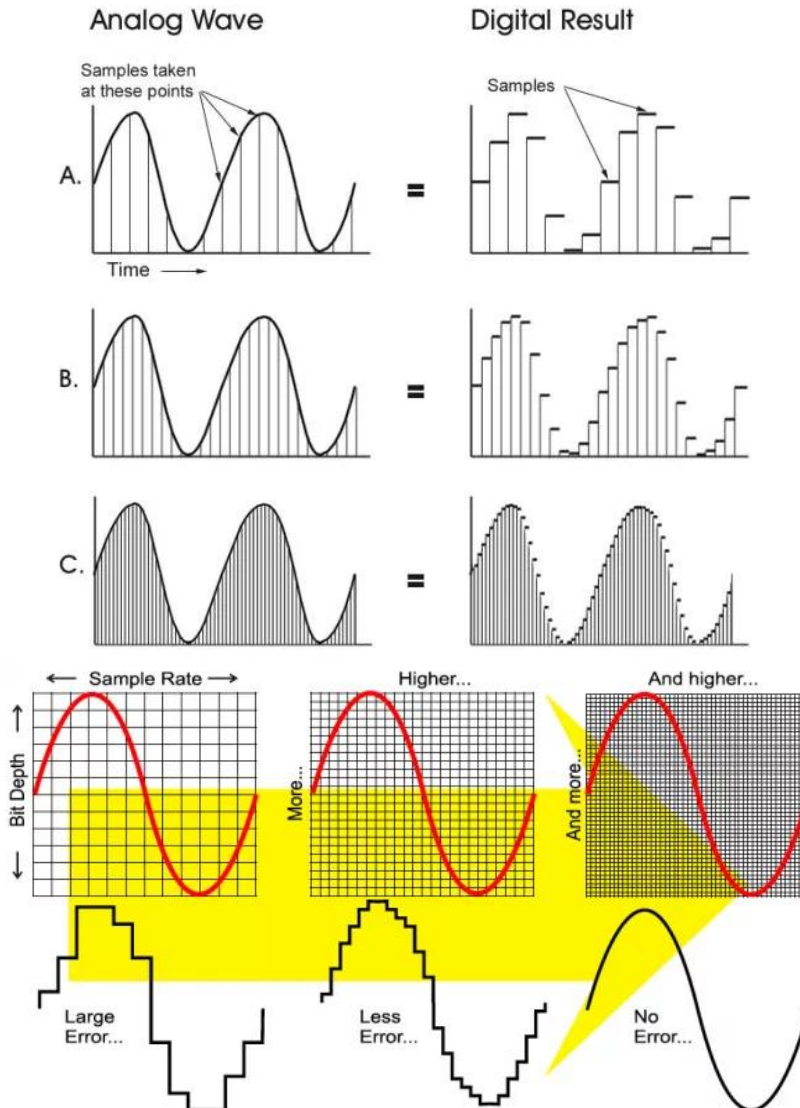


- Images are created using primitive geometric shapes defined on a Cartesian plane, such as:
 - Points
 - Lines
 - Curves
 - Polygons
- Only key parameters of primitive shapes are stored in a markup language.
- Commonly used in *graphic design and illustrations, GIS, CAD and 3D graphics and animation*
- File formats: *SVG, AI, CDR, CGM, WMF*

Representing Sound

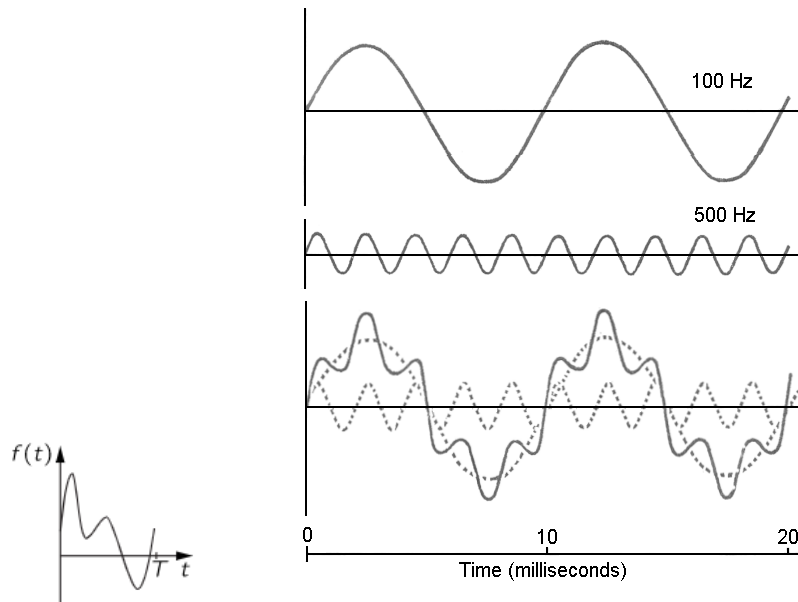
- Sampling techniques
 - Used for high quality recordings
 - Records actual audio
- MIDI
 - Used in music synthesizers
 - Records “musical score”

Recorded Sound Waves



- Amplitudes of an analogue soundwave are sampled at a constant *sample rate* of 44,100 Hz.
- Humans can hear between 20 Hz to 20,000 Hz range.
 - For every cycle of the wave, we need to measure at least 2 amps: positive and negative.
 - Therefore, $2 \times 20\text{kHz} = 40\text{kHz}$.
- *Bit depth* determines the range of each amplitude measurement.
 - 16-bit depth covers 65,536 amp. values
 - 24-bit depth covers 16,777,217 amp. values
 - 32-bit depth covers 4,284,967,296 amp. values
- Formats: mp3, wav, aiff, ogg, wma

Synthesized Sound Waves



$$= a_0$$

$$+ a_1 \cos \omega t$$

$$+ a_2 \cos 2\omega t$$

+ etc.

$$+ b_1 \sin \omega t$$

$$+ b_2 \sin 2\omega t$$

+ etc.

$$f(t) = a_0$$

$$+ a_1 \cos \omega t + b_1 \sin \omega t$$

$$+ a_2 \cos 2\omega t + b_2 \sin 2\omega t$$

$$+ a_3 \cos 3\omega t + b_3 \sin 3\omega t$$

$$+ \dots + \dots$$

- Fourier discovered that complex soundwaves can be approximated to a sum of sine and cosine waves at different *amplitude, frequency* and *phase*.
- Pythagoras found music note is the linear combination of its fundamental frequency and its natural fractions (overtones) represented as *harmonic series*.
- This model can be used to reproduce the sound of different musical instruments by *additive synthesizing*.
- Storing key information about *amplitude, frequency* and *phase* for each overtone enables us storing and playing electronic music.

Representing Numeric Values

- **Binary notation:** Uses bits to represent a number in base two instead of ASCII codes.
- For example:
 - To store 25, we need 2 bytes.
 - Maximum number we can store in 2 bytes with ASCII codes is 99
 - However with 2 bytes (16 bits), it is possible to write $2^{16} = 65536$ distinct numbers in binary notation.
 - Therefore it is efficient to store integers (whole numbers) in binary notation, rather than with ASCII chars.

The Binary System

The traditional decimal system is based on powers of ten.

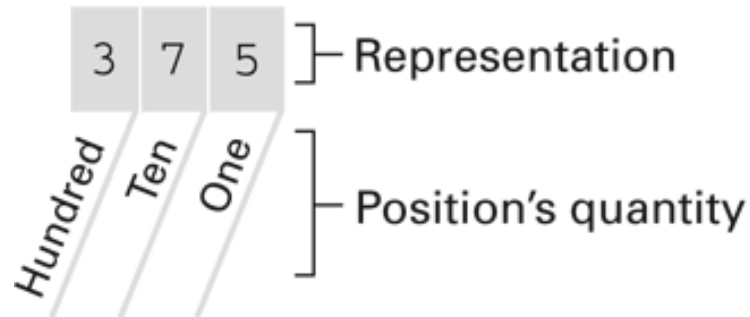
$$375 = 3 * 10^2 + 7 * 10^1 + 5 * 10^0$$

The Binary system is based on powers of two.

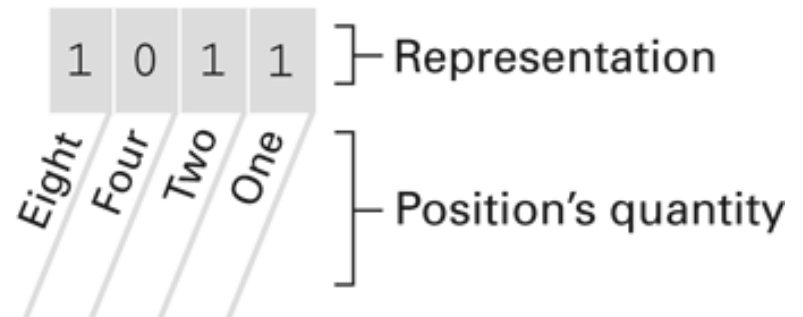
$$11 = \mathbf{1} * 2^3 + \mathbf{0} * 2^2 + \mathbf{1} * 2^1 + \mathbf{1} * 2^0$$
$$(11)_{10} = (1011)_2$$

Base 10 and binary (base 2) systems

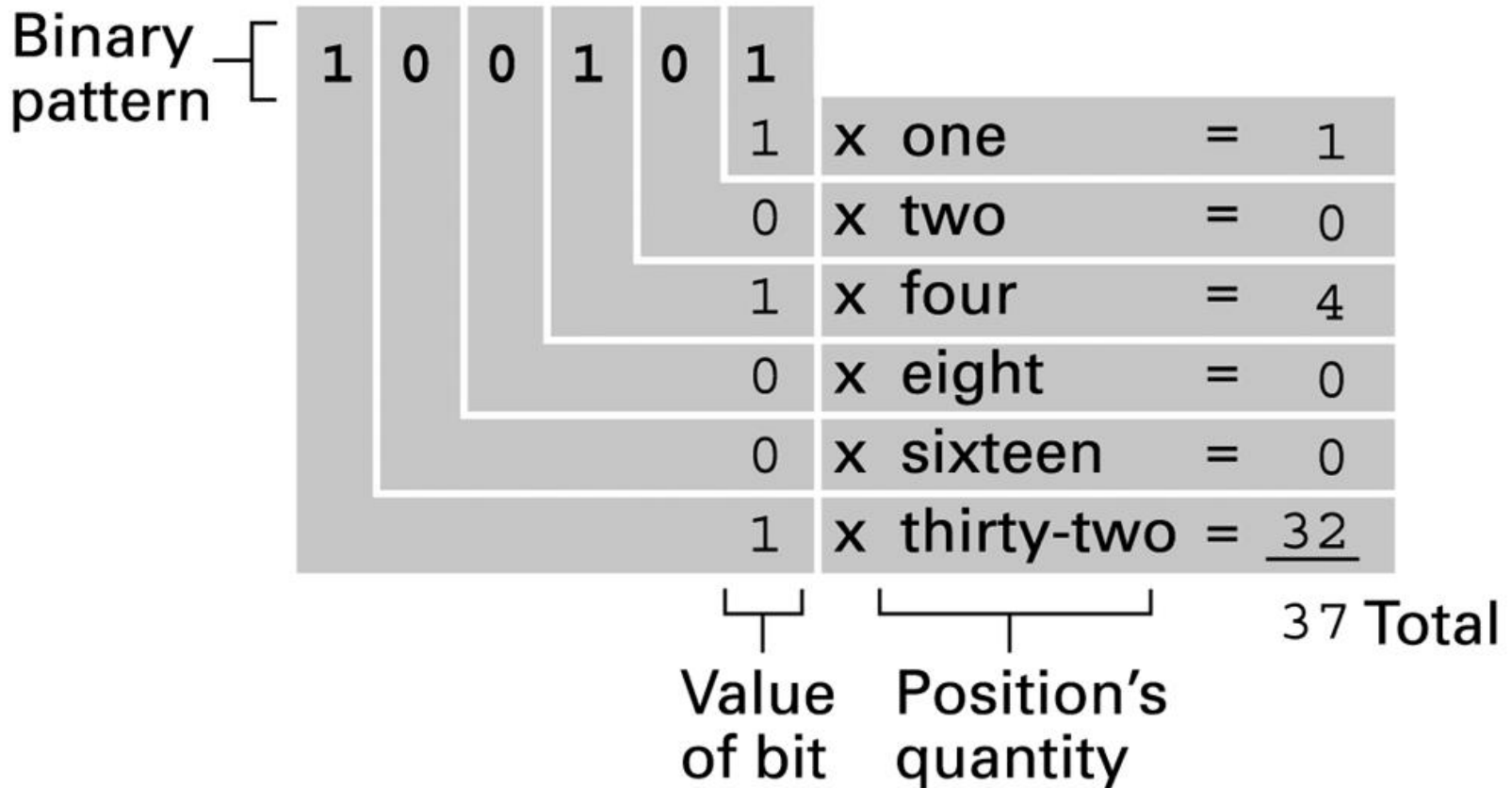
a. Base ten system



b. Base two system



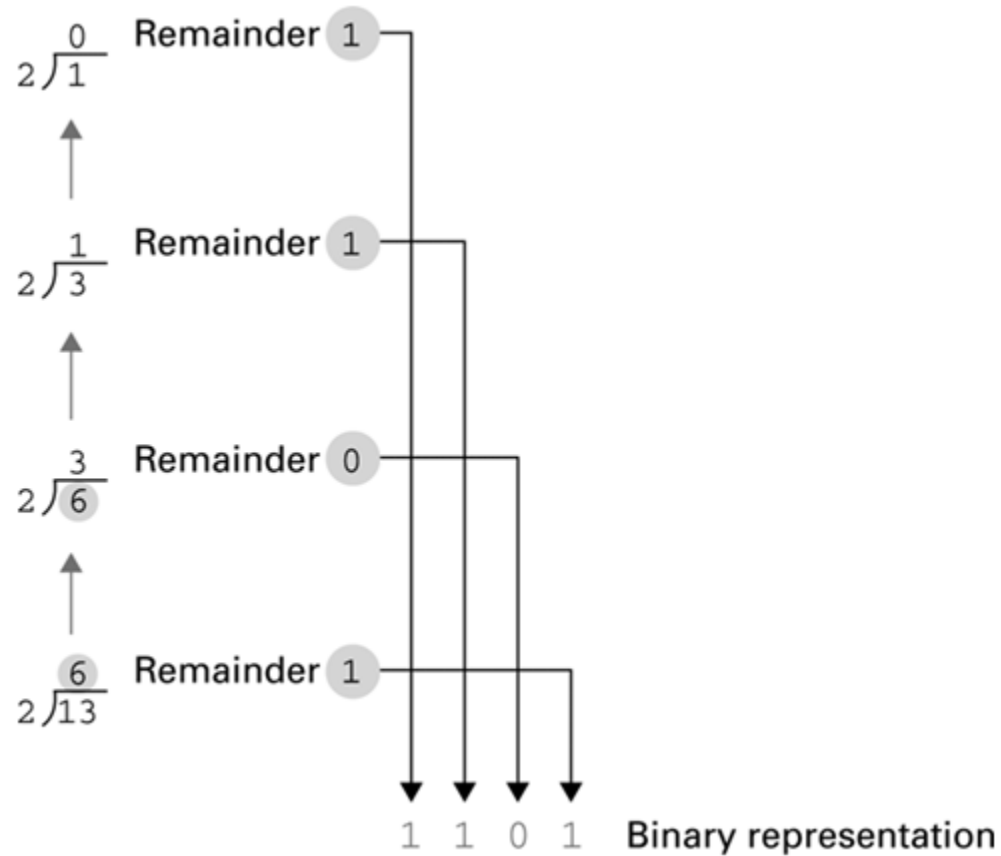
Converting binary to base 10



Converting base 10 to binary

- Step 1.** Divide the value by two and record the remainder.
- Step 2.** As long as the quotient obtained is not zero, continue to divide the newest quotient by two and record the remainder.
- Step 3.** Now that a quotient of zero has been obtained, the binary representation of the original value consists of the remainders listed from right to left in the order they were recorded.

Converting base 10 to binary



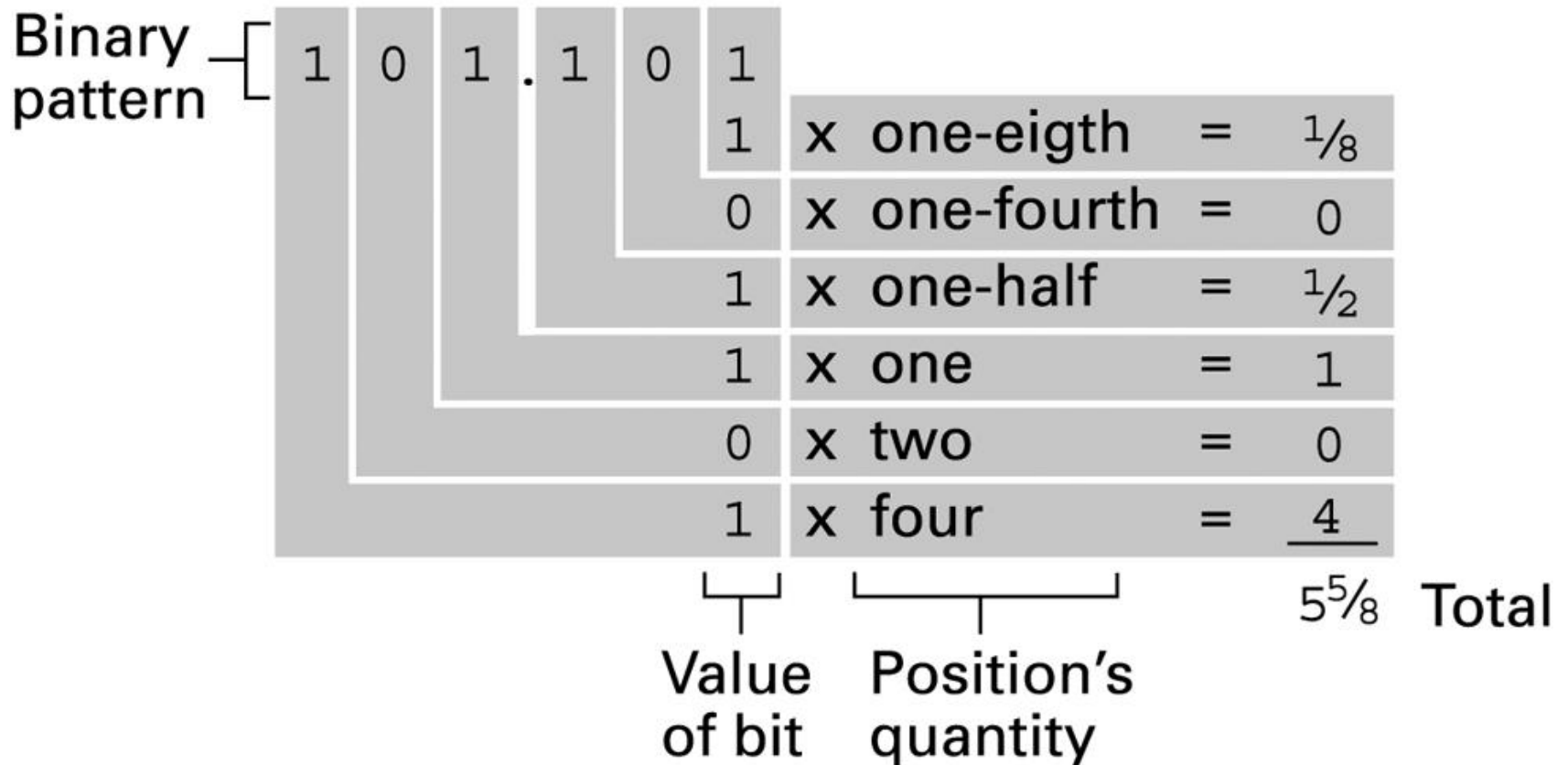
Binary Arithmetic

- ❑ Modern computers usually use binary representation of numerical data, therefore, numerical calculations must be done in radix-2 as well
- ❑ Binary calculations are simple, but remembering a long sequence of binary numbers is the difficult part (good thing that computers have photographic memory):

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \end{array} \quad \begin{array}{r} 1 \\ + 0 \\ \hline 1 \end{array} \quad \begin{array}{r} 0 \\ + 1 \\ \hline 1 \end{array} \quad \begin{array}{r} 1 \\ + 1 \\ \hline 10 \end{array}$$

$$\begin{array}{r} 111010 \\ + 11011 \\ \hline 1010101 \end{array}$$

Binary Representation of Fractions



Converting fractions to binary representation (fixed point notation)

- To convert $\frac{5}{8}$ to binary digits 0.101, you can inverse the above-mentioned process:

$$\frac{5}{8} \times 2 = \frac{5}{4} = 1 + \frac{1}{4} \rightarrow \frac{5}{8} = 1 \times 2^{-1} + 2^{-1} \times \frac{1}{4}$$

$$\frac{1}{4} \times 2 = \frac{1}{2} = 0 + \frac{1}{2} \rightarrow \frac{1}{4} = 0 \times 2^{-1} + 2^{-1} \times \frac{1}{2}$$

$$\rightarrow \frac{5}{8} = 1 \times 2^{-1} + 2^{-1} \times (0 \times 2^{-1} + 2^{-1} \times \frac{1}{2})$$

$$\rightarrow \frac{5}{8} = 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

Exercises

- Convert 101010 to base 10?
- Convert 96 to base 2?
- Convert $5 \frac{5}{8}$ to base 2?
- Convert 11.01 to base 10?
- Convert 23.625 to base 2?
- Convert 1.1 to base 2?

Storing Integers

- **Two's complement notation:** The most popular means of representing integer values
- **Excess notation:** Another means of representing integer values
- Both can suffer from overflow errors

Two's complement notation systems

- Most significant bit represents the sign. 0 means positive, 1 is negative.
- Uses **copy and complement** algorithm to flip signs.

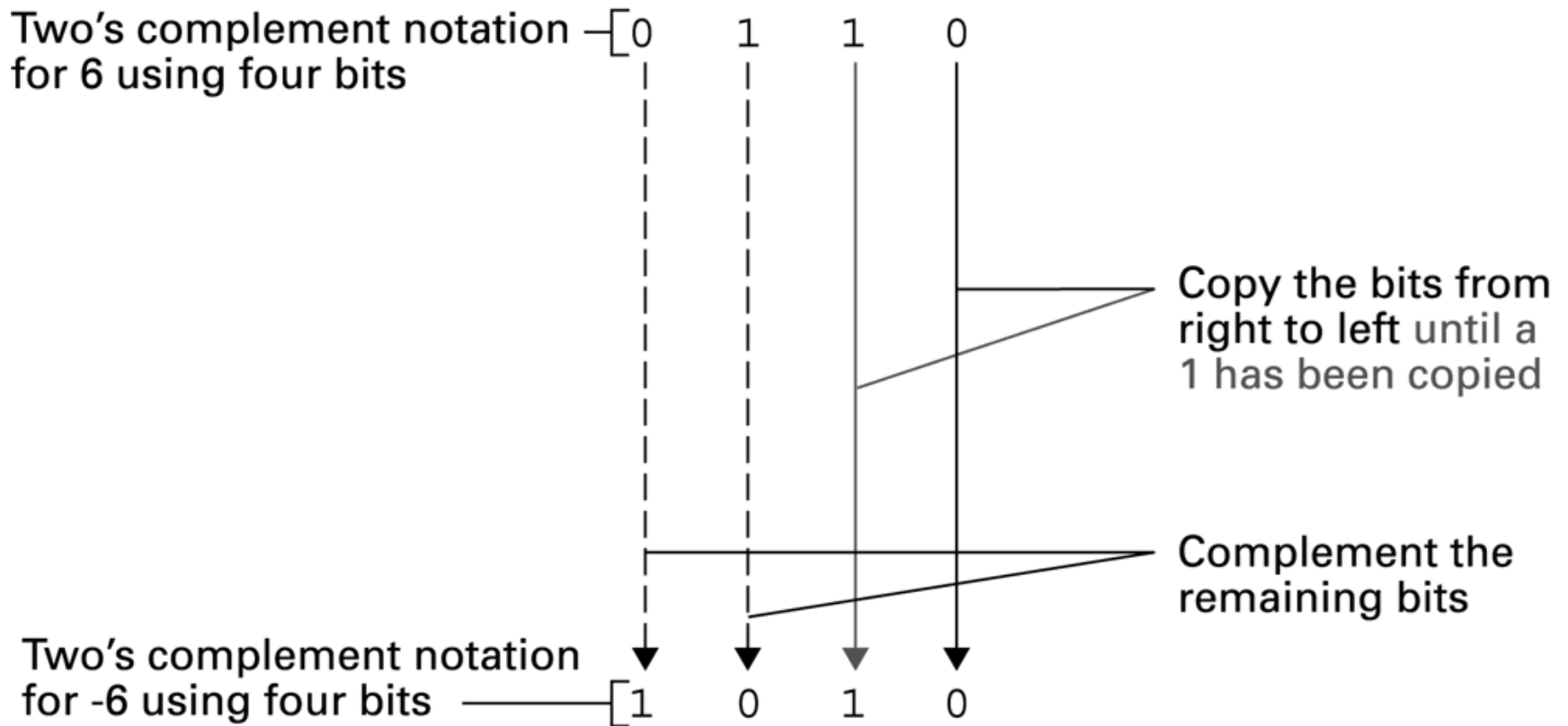
a. Using patterns of length three

Bit pattern	Value represented
011	3
010	2
001	1
000	0
111	-1
110	-2
101	-3
100	-4

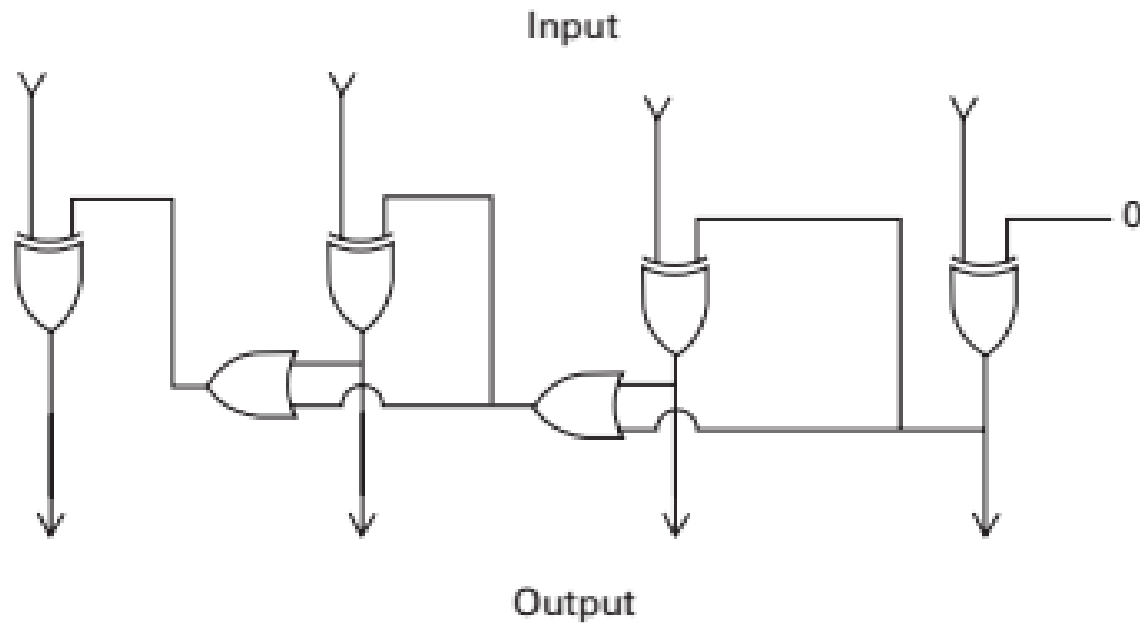
b. Using patterns of length four

Bit pattern	Value represented
0111	7
0110	6
0101	5
0100	4
0011	3
0010	2
0001	1
0000	0
1111	-1
1110	-2
1101	-3
1100	-4
1011	-5
1010	-6
1001	-7
1000	-8

Coding the value -6 in two's complement notation using four bits



2's complement negating circuit



Addition problems converted to two's complement notation

- Sum of 2 integers in two's complement notation always results in an integer with the same length.
- Any exceeding digits are simply truncated.
- Using two's complement notation, it is possible to subtract 2 positive numbers by simply converting the second number to negative, and adding them together.
- For ex, $7-5=7+(-5)$

Problem in base ten		Problem in two's complement		Answer in base ten
$\begin{array}{r} 3 \\ + 2 \\ \hline \end{array}$	→	$\begin{array}{r} 0011 \\ + 0010 \\ \hline 0101 \end{array}$	→	5
$\begin{array}{r} -3 \\ + -2 \\ \hline \end{array}$	→	$\begin{array}{r} 1101 \\ + 1110 \\ \hline 1011 \end{array}$	→	-5
$\begin{array}{r} 7 \\ + -5 \\ \hline \end{array}$	→	$\begin{array}{r} 0111 \\ + 1011 \\ \hline 0010 \end{array}$	→	2

Overflow Problem

- Because in 4-bit two's complement can represent numbers from -8 to 7, any operation resulting lower or higher number will cause **overflow error**.
- Hence, $5+4$ results -7, which is incorrect.
- Overflow can be detected if:
 - Sum of 2 positives result in negative number
 - Sum of 2 negatives result in positive number
- Modern computers store integers in 32 bits, hence making any arithmetic operation not exceeding 2 147 483 647 possible.

An excess eight conversion table

- All numbers are shifted by a constant.
- 4-bit excess notation shifts numbers by 8.
- Unlike 2's complement, the most significant bit for positive number is 1.

So, for n-bit digit, we can compute its excess as $2^{(n-1)}$

For ex, 11001 is a 5 digit number, its excess is $2^4=16$. So, $11001_2 = 25_{10} = (25 - 16)_e = 9_e$

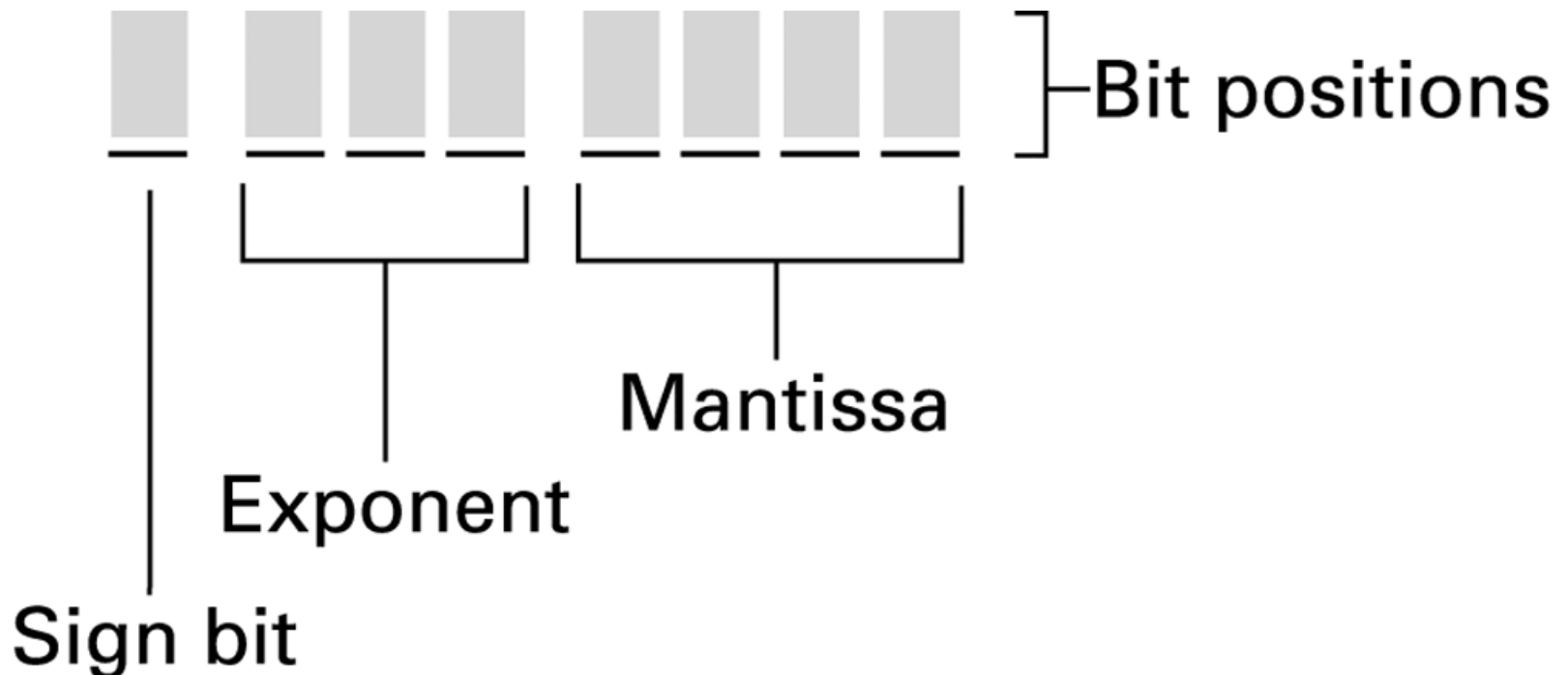
Bit pattern	Value represented
1111	7
1110	6
1101	5
1100	4
1011	3
1010	2
1001	1
1000	0
0111	-1
0110	-2
0101	-3
0100	-4
0011	-5
0010	-6
0001	-7
0000	-8

Exercises

- Convert from 2's complement to base 10:
00011, 11100, 11010
- Convert from base 10 to 2's complement:
6, -17, -1
- Compute:
0100+0011, 1010+0111
- Convert from excess-8 to base 10:
1110, 0111

Storing Fractions

- **Floating-point Notation:** Consists of a sign bit, a mantissa field, and an exponent field.

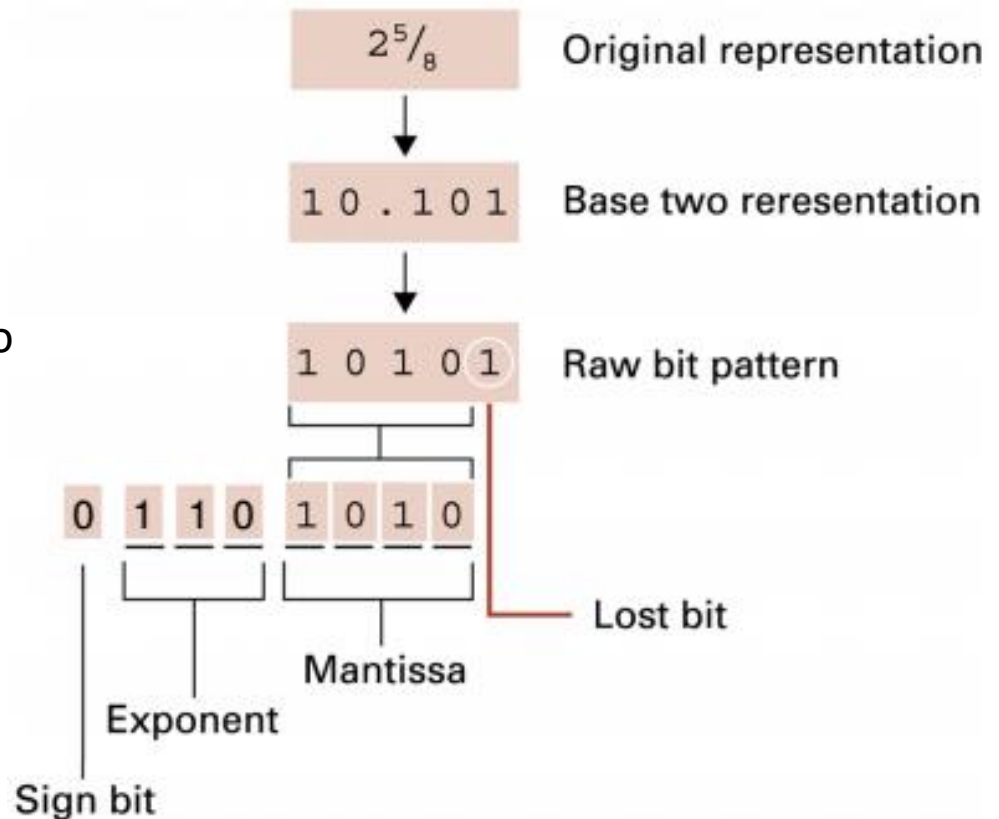


Encoding the value $2^{5/8}$

- Sign bit stores the sign of the information.
- Exponent tells us where to shift the floating point:
 - If it is positive, move right to given number of positions
 - If it is negative, move left to given number of positions
- Mantissa stores the raw bits of the number.

Important 1: Exponent part is written in excess-notation.

Important 2: Precision can be lost due to limitation in mantissa bits



Normalized Form

- Mantissa stores the raw bits starting with the leftmost 1.
- It ignores the leading zeros.
- For ex, in 4 bit mantissa, 0110, is stored as 1100
- Normalized form saves more space and eliminates the possibility of multiple representations of the same value.
- For ex, 00111100 and 01000110 would have produced $3/8$

Truncation Errors

- Truncation errors happen because there is not enough bits in mantissa to store all numbers.
- For example,

$$2.5 + 0.125 + 0.125$$

- If added from left to right, produce 2.5
- If added from right to left, produce 2.75
- Truncation errors are common even in today's computers, and they usually happen when adding large numbers with small numbers.
- Therefore numerical analysts first sum small numbers in hope that it results in large number.

Types of Floating Point in Your Computer

- **Single Precision Floating Point** - 32 bits, can represent numbers as large as 10^{38} and fractions as tiny as 10^{-37} with precision upto 7 decimal places.
 - Sign – 1 bit
 - Exponent – 8 bits
 - Mantissa – 23 bits
- **Double Precision Floating Point** - 64 bits, can provide precision up 15 decimal places.
 - Sign – 1 bit
 - Exponent – 11 bits
 - Mantissa – 53 bits

Data Compression

- Lossy versus lossless:
 - Lossy lose information while compressed
 - Lossless do not lose information
- Run-length encoding (RLE) – lossless compression which simply substitutes repeating sequences of same elements with their quantity and element.
For ex,
 - “FFFFFFFF” can be represented as 8F, meaning I have to insert 8 “F”s.
- Frequency-dependent encoding, unlike unicode where ever character has equal number of bits, in this compression common chars have less bits, and less common have more bits.
 - For ex, in English letters “e”, “t”, “a” are used more commonly than “z”, “q”, and “x”. So the common letters can be represented by fewer bits, and thus reduce the size.
 - The algorithm for generating frequency-based codes has been developed by David Huffman, and their output is referred as Huffman codes.

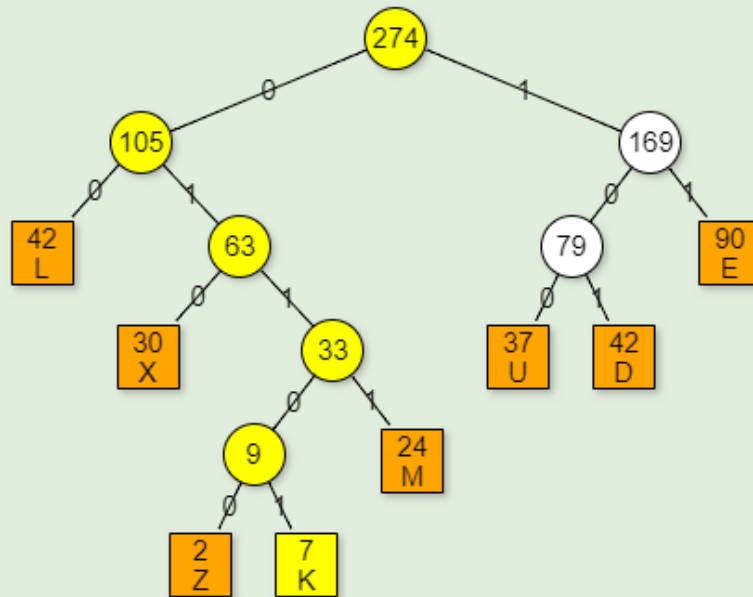
Huffman Codes

frequency of letters in text

30 X	42 D	90 E	7 K	42 L	24 M	37 U	2 Z
---------	---------	---------	--------	---------	---------	---------	--------



Huffman code for character "K" is: "01101", which is the value along the path from the root "274" to "7"



Data Compression

- **Relative encoding** (Differential encoding) stores only the first element, and for the subsequent elements it store the difference between them.
 - This method of compression has wide application in audio/video compression algorithms
- **Dictionary encoding** algorithm uses either fixed/dynamic dictionary of words to substitute them with values of less size.
 - Lempel-Ziv-Welsh (LWZ) algorithm uses dynamic dictionary, it starts off with initial small dictionary and grow it with more values in the process of compression
 - While decompressing, LWZ does not need to have the final dictionary, it can construct from initial small dictionary in the process.

Compressing Images

- GIF: Good for cartoons, lossy
 - Maintains fixed dictionary of 256 colors, and maps every color to it.
 - Has also 'transparent' color
- JPEG: Good for photographs, lossy or lossless
 - Uses multistep averaging techniques to reduce the size of the picture
 - Plays on human perception of brightness and chrominance
 - Chrominance can be aggregated for 8-by-8 pixel block, while brightness data is kept intact.
 - Instead of storing absolute color values, relative differences between subsequent blocks can be stored.
- TIFF: Good for document scans
 - Uses standard Run-Length compression, good for texts with much whitespace.

Compressing Audio and Video

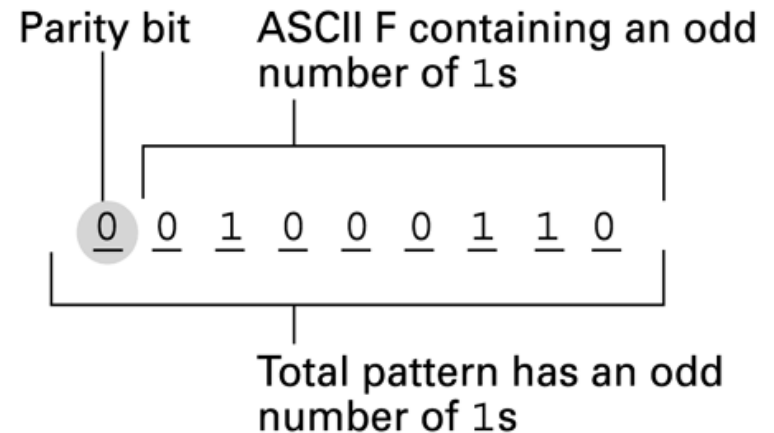
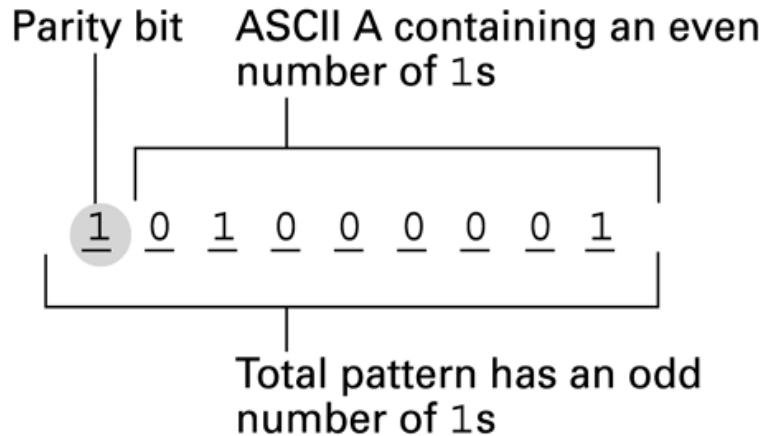
- Video compression:
 - **Intraframe compression:** each frame is compressed using image compression algorithm
 - **Interframe compression:** differences between each frame can be transmitted to reconstruct absolute pixels for each frame from given pivot frame (aka I-frame).
- Audio compression:
 - **Temporal masking** - when loud sound is played humans do not hear any other sound which is lower in volume within 200ms timeframe. Therefore it is reasonable not to digitize any sound in the vicinity of loud sounds for certain interval of time.
 - **Frequency masking** - when sound waves with similar frequencies are played together, humans do not distinguish the difference between them, so it is worth eliminating redundant parallel sound waves.

Error Protection

- Computing systems are not always reliable. Numbers stored in a computing system can have random bit errors over time.
- To detect and/or correct errors, the number representation has to be robust to random bit errors
- Two types of error-robust coding possible:
 - Error detection coding:
 - Parity bits (even versus odd)
 - Checkbytes
 - Error correcting codes
 - Generating bit-patterns with minimum Hamming distance

Error Detection

Parity bits: the ASCII codes for the letters A and F adjusted for odd parity



Checksum is the collection of parity bits stored separately. Each bit in checksum corresponds to a parity bit of some pattern. For example, first bit may represent a parity bit for every 8th bit in given block of data, second bit for other bit pattern, etc.

Error Correction

Hamming distance: Number of edits (insertions, deletions or replacements) required to transform one pattern to another.

An error-correcting code

Symbol	Code
A	000000
B	001111
C	010011
D	011100
E	100110
F	101001
G	110101
H	111010

Decoding the pattern 010100

Character	Code	Pattern received	Distance between received pattern and code	
A	0 0 0 0 0 0	0 1 0 1 0 0	2	
B	0 0 1 1 1 1	0 1 0 1 0 0	4	
C	0 1 0 0 1 1	0 1 0 1 0 0	3	
D	0 1 1 1 0 0	0 1 0 1 0 0	1	Smallest distance
E	1 0 0 1 1 0	0 1 0 1 0 0	3	
F	1 0 1 0 0 1	0 1 0 1 0 0	5	
G	1 1 0 1 0 1	0 1 0 1 0 0	2	
H	1 1 1 0 1 0	0 1 0 1 0 0	4	

Suggested Activity

- Synthesize Sound Waves
 - <https://meettechnik.info/additional/additive-synthesis.html>
- Visualize Huffman Coding
 - <https://cmps-people.ok.ubc.ca/ylucet/DS/Huffman.html>
 - Implement Huffman Coding algorithm in Python to encode/decode your messages
- Watch CS101 Crash Course videos
 - <https://www.youtube.com/watch?v=1GSjbWt0c9M&list=PL8dPuuaLjXtNIUrzyH5r6jN9uIlgZBpdo&index=5>